

Споредба на класичен и SAT-базиран пристап за проблемот Independent Set

Ивана Пенкова 226021, Евгенија Арсова 226046, Александра Станојковска 226016

Факултет за информатички науки и компјутерско инженерство
Универзитет „Св. Кирил и Методиј“ во Скопје, Република Северна Македонија
{ivana.penkova, evgenija.arsova, aleksandra.stanojkovska}@students.finki.ukim.mk

1 Вовед

Проблемот *Independent Set* (Независно множество) е еден од позначајните комбинаторни проблеми од теоријата на графови и теоријата на пресметувачка сложеност. Кај овој проблем, за даден неориентиран граф потребно е да се избере множество од темиња такво што ниту едни две избрани темиња не се меѓусебно соседни.

Decision верзијата на проблемот дополнително задава цел број k и го поставува прашањето дали во графот постои независно множество со најмалку k темиња. Independent Set припаѓа во класата NP и претставува NP-комплетен проблем, поради што е погоден за анализа и споредба на различни пристапи за решавање.

Целта на овој проект е експериментално да се споредат два различни пристапи за решавање на проблемот. Првиот е директен класичен пристап базиран на рекурзивно backtracking пребарување. Вториот пристап го трансформира Independent Set проблемот во SAT формула во CNF форма, која потоа се решава со SAT решавачи.

Во рамки на SAT пристапот се користат два решавачи, **MiniSat** и **CaDiCaL**. На тој начин, покрај споредбата помеѓу класичниот и SAT-базираниот пристап, се овозможува и споредба помеѓу два SAT решавачи врз истото енкодирање.

Експерименталната анализа се изведува врз автоматски генерирани графови со различен број на темиња, различна веројатност за генерирање на ребро и различна барана големина на независното множество. Се мерат времињата на извршување, времето потребно за SAT енкодирање и решавање, како и големината на добиените CNF формули.

Целта е да се утврди како карактеристиките на влезната инстанца влијаат врз перформансите и во кои ситуации секој од пристапите е поповолен.

2 Формална дефиниција на проблемот

Нека е даден неориентиран граф

$$G = (V, E),$$

каде што V е множество од темиња, а E е множество од ребра.

Множество $S \subseteq V$ се нарекува *независно множество* ако ниту едни две различни темиња од S не се поврзани со ребро. Формално:

$$\forall u, v \in S, \quad u \neq v \Rightarrow (u, v) \notin E.$$

Во овој проект се разгледува decision верзијата на проблемот Independent Set.

2.1 Влез

Влезот се состои од:

- неориентиран граф $G = (V, E)$;
- позитивен цел број k .

2.2 Излез

Излезот е Булова вредност:

YES

ако постои независно множество $S \subseteq V$ такво што

$$|S| \geq k,$$

или

NO

ако такво множество не постои.

Според тоа, decision проблемот може формално да се запише како:

$$\text{IS} = \{(G, k) \mid \exists S \subseteq V, |S| \geq k, \\ \forall u \neq v \in S : (u, v) \notin E\}.$$

каде што IS ја означува decision верзијата на Independent Set.

2.3 Класа на сложеност

Independent Set припаѓа во класата NP, бидејќи за дадено кандидатско множество S може во полиномско време да се провери дали $|S| \geq k$ и дали не постои ребро помеѓу две темиња од S .

NP-тежината може директно да се види преку стандардната врска со VERTEX COVER. За секој граф $G = (V, E)$ важи дека $S \subseteq V$ е независно множество ако и само ако $V \setminus S$ е vertex cover. Затоа, за инстанца (G, t) на VERTEX COVER важи

Ако со VC и IS ги означиме двата decision проблема, тогаш

$$(G, t) \in \text{VC} \iff (G, |V| - t) \in \text{IS}.$$

Ова е полиномска редукција, а VERTEX COVER е меѓу класичните NP-комплетни проблеми [1]. Заедно

со членството во NP, следува дека Independent Set е NP-комплетен. Поради тоа, за поголеми влезни инстанци не се очекува алгоритам што во општ случај го решава проблемот во полиномско време, освен ако $P = NP$.

3 Класичен алгоритам

Како класичен пристап за решавање на Independent Set проблемот е имплементиран рекурзивен backtracking алгоритам.

Алгоритмот ги разгледува темињата едно по едно и за секое теме има две можности:

- темето да биде вклучено во тековното независно множество;
- темето да биде прескокнато.

Пред едно теме да биде додадено во тековното решение, се проверува само дали тоа е соседно со некое од веќе избраните темиња. На тој начин не се врши повторна целосна валидација на истото парцијално решение во секој рекурзивен чекор. Доколку постои конфликтно ребро, темето не може да биде додадено во независното множество.

Пребарувањето завршува успешно веднаш штом се најдат најмалку k меѓусебно несоседни темиња.

3.1 Backtracking постапка

Нека *selected* е тековното множество на избрани темиња, а *index* позицијата на следното теме што треба да се разгледа.

За секое теме v алгоритмот најпрво се обидува да го вклучи во решението. Ако множеството

$$selected \cup \{v\}$$

останува независно, пребарувањето рекурзивно продолжува со тоа множество.

Ако таквото пребарување не доведе до решение, алгоритмот се враќа назад и ја разгледува втората можност, односно тековното теме да не биде избрано.

На овој начин се истражуваат различни комбинации на темиња сè додека не се најде решение или не се исцрпат сите релевантни можности.

3.2 Pruning

За намалување на непотребното пребарување се користи едноставна pruning техника.

Ако во одреден момент важи:

$$|selected| + |remaining| < k,$$

тогаш дури и ако сите преостанати темиња бидат избрани, не може да се достигне големина k .

Во таков случај гранката веднаш се прекинува.

3.3 Временски лимит

Кај поголемите и потешките инстанци backtracking алгоритмот може да има многу долго време на извршување. Поради тоа, во експерименталниот дел е

поставен временски лимит од 10 секунди по извршување.

Доколку алгоритмот не заврши во рамките на овој период, инстанцата се означува како *timeout* и експериментот продолжува со SAT пристапите.

3.4 Временска сложеност

Во најлош случај, за секое теме се разгледуваат две можности, дали да биде избрано или не. Поради тоа, бројот на можни комбинации може да расте експоненцијално со бројот на темиња.

Приближната worst-case сложеност на основниот backtracking пристап е:

$$O(2^{|V|}),$$

при што pruning техниките можат значително да го намалат реалниот број на разгледани состојби, но не ја менуваат експоненцијалната worst-case природа на алгоритмот.

4 SAT енкодирање

Вториот пристап го трансформира Independent Set проблемот во SAT формула во конјунктивна нормална форма (CNF). Потоа истата CNF формула се решава со два различни SAT решавачи, MiniSat [2] и CaDiCaL [3].

Целта на енкодирањето е формулата да биде задоволена ако и само ако дадениот граф содржи независно множество со големина најмалку k .

4.1 Булови променливи

За секое теме $v_i \in V$ се дефинира една Булова променлива

$$x_i.$$

Нејзиното значење е:

$$x_i = \begin{cases} 1, & \text{ако темето } v_i \text{ е избрано;} \\ 0, & \text{ако темето } v_i \text{ не е избрано.} \end{cases}$$

На овој начин изборот на темиња во независното множество се претставува со вистинитосни вредности на SAT променливите.

4.2 Ограничувања за независност

Основниот услов на Independent Set е две соседни темиња да не можат истовремено да бидат избрани.

За секое ребро

$$(v_i, v_j) \in E$$

се додава CNF клаузулата

$$\neg x_i \vee \neg x_j.$$

Оваа клаузула е неточна единствено ако истовремено важи

$$x_i = 1 \quad \text{и} \quad x_j = 1,$$

односно ако двете соседни темиња се избрани.

Со додавање на ваква клаузула за секое ребро во графот се гарантира дека сите избрани темиња формираат независно множество.

4.3 Ограничување за големината

Самите ограничувања за ребрата не се доволни. На пример, доделувањето во кое сите променливи се *false* секогаш би ги задоволито клаузулите за независност.

Поради тоа дополнително се поставува условот дека мора да бидат избрани најмалку k темиња:

$$\sum_{i=1}^{|V|} x_i \geq k.$$

Во имплементацијата овој cardinality услов се претвора во CNF со помош на `CardEnc.atleast` од библиотеката PySAT [4, 5]. Експлицитно се избира `sequential-counter` енкодирање (`EncType.seqcounter`), односно варијанта на `sequential-counter` пристапот за cardinality ограничувања [6]. PySAT го реализира `AtLeastK` условот преку соодветен `AtMost` услов над негираниите литерали.

При оваа трансформација, покрај основните променливи што ги претставуваат темињата, се создаваат и помошни SAT променливи. Оригиналните променливи $1, \dots, n$ остануваат резервирани за темињата, а помошните променливи добиваат поголеми идентификатори. При реконструкција на независното множество од SAT моделот се разгледуваат само оригиналните n променливи.

4.4 Целосна SAT формула

Целосната формула може да се претстави како

$$F(G, k) = F_{\text{edges}} \wedge F_{\text{cardinality}}.$$

каде што F_{edges} ги содржи ограничувањата за сите ребра, а $F_{\text{cardinality}}$ го претставува условот за избор на најмалку k темиња.

За коректното енкодирање важи:

$F(G, k)$ е SAT $\iff G$ содржи независно множество со големина најмалку k .

Ако SAT решаваачот врати *SAT*, од моделот се издвојуваат оние оригинални променливи x_i што имаат вредност *true*. Со нив се добива множеството на избрани темиња.

Ако решаваачот врати *UNSAT*, тогаш за даденото k не постои независно множество со потребната големина.

4.5 SAT решаваачи

Во проектот се користат два SAT решаваачи:

- **MiniSat 2.2**, како класичен SAT решаваач;
- **CaDiCaL**, како понов SAT решаваач.

Двата решаваачи ја добиваат истата CNF формула. На овој начин SAT енкодирањето останува непроменето, а при споредбата се менува само решаваачот.

Ова овозможува пофер споредба на нивните перформанси, бидејќи разликите во времето на решавање не се последица на различно енкодирање.

Во експериментите дополнително се мери бројот на CNF променливи и бројот на CNF клаузули, како и времето потребно за генерирање и решавање на формулата.

5 Експериментална поставеност

Целта на експерименталниот дел е да се споредат перформансите на класичниот `backtracking` пристап и SAT-базираните пристапи со MiniSat и CaDiCaL при различни карактеристики на влезниот граф.

За сите методи се користат истите графови и истите вредности на параметарот k , со што се обезбедува директна и фер споредба.

5.1 Генерирање на графови

За тестирање се користат случајно генерирани неориентирани графови според моделот $G(n, p)$, каде што:

- n е бројот на темиња;
- p е веројатноста да постои ребро помеѓу секој пар различни темиња.

За секој можен пар темиња (u, v) , ребро се додава независно со веројатност p . Важно е p да се толкува како *edge-generation probability*, а не како точната реализирана густина на конкретниот граф. Реализираниот број на ребра варира од `seed` до `seed`, иако неговата очекувана вредност е определена од p .

Во финалните експерименти се користат следните вредности:

$$n \in \{20, 30, 40, 50, 60\},$$

$$p \in \{0.2, 0.5, 0.8\}.$$

Со вредноста $p = 0.2$ се очекуваат релативно ретки графови, $p = 0.5$ графови со средна очекувана густина, а $p = 0.8$ густо графови.

За да може експериментите повторно да се извршат врз истите графови, се користат фиксни `random seed` вредности:

$$\text{seed} \in \{1, 2, 3\}.$$

Генераторот користи локална `random` состојба, така што генерирањето на една инстанца не ја менува глобалната `random` состојба на програмата.

5.2 Избор на параметарот k

Покрај бројот на темиња и густината на графот, се менува и бараната големина на независното множество.

Параметарот k се определува како дел од вкупниот број на темиња:

$$\frac{k}{n} \in \{0.3, 0.4\}.$$

Во имплементацијата се користи

$$k = \max(1, \lfloor n \cdot r \rfloor), \quad r \in \{0.3, 0.4\}.$$

За избраните вредности на n производите се цели броеви, на пример, за $n = 50$ се добиваат $k = 15$ и $k = 20$.

Комбинирањето на сите вредности за n , p , random seed и k резултира со:

$$5 \cdot 3 \cdot 3 \cdot 2 = 90$$

различни тест инстанци.

5.3 Мерење на времето

Времето на извршување се мери со функцијата `perf_counter()` од Python.

За да се намали влијанието на краткотрајни системски осцилации, секое мерење се повторува пет пати. Како репрезентативно време за конкретна инстанца и метод се користи медијаната од петте измерени вредности. Подоцнежните агрегирани табели повторно користат медијана над овие репрезентативни времиња за сите инстанци што припаѓаат во иста група (на пример, исто n или исто p).

Кај класичниот пристап се мери вкупното време потребно за извршување на backtracking алгоритмот.

Кај SAT пристапот одделно се мерат:

- времето потребно за SAT енкодирање;
- времето потребно за работа на SAT решавачот;
- вкупното време на SAT пристапот.

Вкупното SAT време се пресметува како:

$$T_{\text{SAT}} = T_{\text{encoding}} + T_{\text{solver}}.$$

На овој начин споредбата со backtracking алгоритмот го зема предвид и дополнителниот трошок потребен за трансформација на Independent Set инстанцата во CNF формула.

5.4 Временски лимит

Поради експоненцијалната worst-case сложеност на backtracking алгоритмот, за секое негово извршување е поставен временски лимит од 10 секунди.

Ако алгоритмот не заврши во зададениот период, инстанцата се означува како *timeout*. Timeout се третира одделно од математичкиот исход *YES/NO*: тој не значи дека независно множество не постои, туку само дека конкретното извршување не завршило во лимитот. Во резултатската датотека ваквите извршувања се бележат со време од 10 секунди, по што експериментот продолжува со SAT пристапите за истата инстанца.

5.5 Дополнителни мерења

Покрај времето на извршување, за SAT енкодирањето се евидентираат и:

- бројот на CNF променливи;
- бројот на CNF клаузули.

Овие вредности овозможуваат да се анализира како расте големината на SAT формулата со зголемување на големината на графот.

Сите резултати автоматски се зачувуваат во CSV датотека за понатамошна статистичка анализа и генерирање графици.

5.6 Проверка на коректноста

Пред спроведување на финалните експерименти, имплементацијата е проверена со автоматски тестови врз мали графови за кои однапред е познат точниот резултат.

Тестовите проверуваат:

- позитивни и негативни случаи за backtracking алгоритмот;
- позитивни и негативни случаи за SAT енкодирањето;
- согласност на резултатите од Backtracking, MiniSat и CaDiCaL.

Дополнително, решавачите се одделени од независен валидатор. За секое вратено кандидат-решение се проверува дали множеството содржи најмалку k темиња, дали нема дупликати, дали сите идентификатори се валидни темиња од графот и дали помеѓу избраните темиња не постои ребро. Валидацијата се извршува по завршувањето на решавачот и не е вклучена во мереното solver време.

Сите осум автоматски тестови успешно поминуваат. Сите вратени позитивни кандидат-решенија во финалниот експеримент ја поминуваат независната проверка на коректност.

6 Резултати и анализа

Финалниот експеримент опфаќа вкупно 90 различни влезни инстанци. За секоја инстанца се извршуваат Backtracking, MiniSat и CaDiCaL, со што CSV датотеката содржи 270 записи за извршување на метод. Од нив, 10 backtracking извршувања завршуваат со timeout; останатите завршени резултати успешно ја поминуваат независната валидација.

6.1 Влијание на големината на графот

Во Табела 1 се прикажани агрегирани медијални вкупни времиња според бројот на темиња. Секој ред ги обединува различните вредности на p , k/n и seed за даденото n .

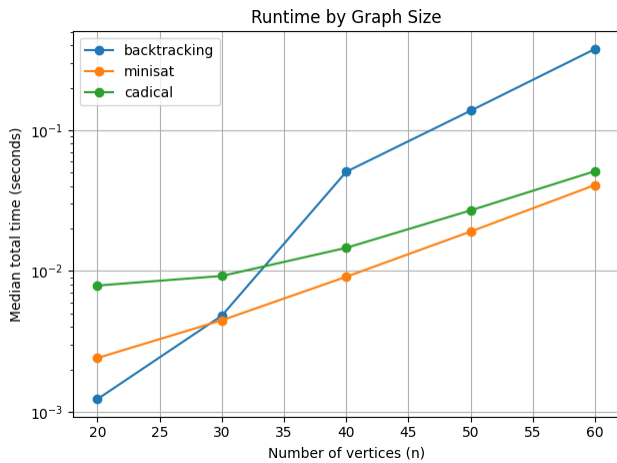
Табела 1: Медијално вкупно време според бројот на темиња

n	Backtracking	MiniSat	CaDiCaL
20	0.001230	0.002409	0.007875
30	0.004815	0.004464	0.009211
40	0.050620	0.009090	0.014587
50	0.137361	0.019043	0.026911
60	0.377058	0.040757	0.051125

Кај најмалите графови со $n = 20$, backtracking има најмало медијално време. Кај овие инстанци директното пребарување е доволно евтино, па дополнителниот трошок за конструирање CNF и иницијализација на SAT решавачот не се исплаќа.

Кај $n = 30$ MiniSat веќе има благо пониско медијално време од backtracking. Со понатамошно зголемување на n разликата станува изразена. Кај $n = 60$, медијалното време на backtracking е околу 0.3771 секунди, додека MiniSat има медијално вкупно време од околу 0.0408 секунди, односно приближно 9.25 пати пониско медијално време во оваа агрегирана група.

Трендот е прикажан на Слика 1. Поради разликите од повеќе редови на големина меѓу дел од мерењата, логаритамската вертикална оска овозможува почитлива споредба.



Слика 1: Медијално вкупно време според бројот на темиња.

6.2 Влијание на edge-generation probability

Параметарот p има силно влијание врз однесувањето на backtracking алгоритмот. Треба да се нагласи дека p е веројатност за генерирање на ребро, а не измерената точна густина на секој поединечен граф.

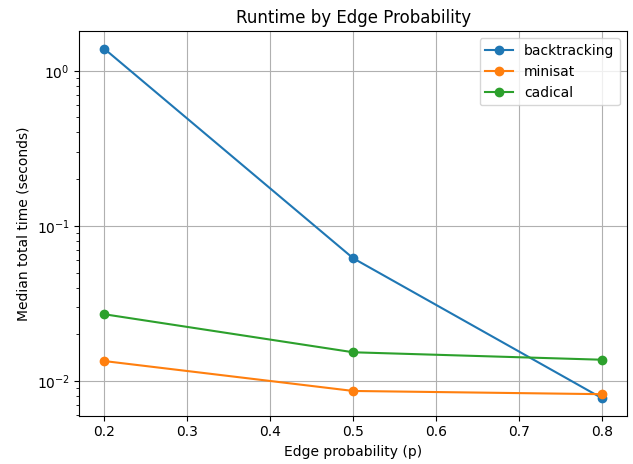
Инстанците генерирани со $p = 0.2$ се покажуваат како најтешки за имплементираниот backtracking алгоритам. Неговото медијално време во оваа група е околу 1.385 секунди, во споредба со околу 0.01345 секунди кај MiniSat. Тоа е приближно 103 пати повисоко медијално време за backtracking во оваа агре-

Табела 2: Медијално вкупно време според параметарот p

p	Backtracking	MiniSat	CaDiCaL
0.2	1.385019	0.013450	0.026941
0.5	0.061811	0.008629	0.015322
0.8	0.007744	0.008226	0.013717

гирана група.

Со зголемување на p , backtracking може порано да отфрла кандидати, бидејќи постојат повеќе конфликтни ребра меѓу темињата. Кај $p = 0.8$ backtracking има медијално време од 0.007744 секунди, а MiniSat 0.008226 секунди; тука директниот пристап е благо побрз. Овој резултат е важен затоа што покажува дека SAT редукцијата не носи автоматска предност кај секој тип инстанца.



Слика 2: Медијално вкупно време според edge-generation probability p .

6.3 Временски ограничувања

Од вкупно 90 инстанци, backtracking го достигнува временскиот лимит во 10 случаи. Сите timeout случаи се кај $p = 0.2$: четири се појавуваат за $n = 50$, а шест за $n = 60$. Кај $n = 60$ и $p = 0.2$ тоа значи timeout за сите шест комбинации на seed и k/n .

Овој резултат ја зајакнува забелешката дека ретките и поголеми графови се критични за конкретната backtracking имплементација. SAT решавачите ги завршуваат сите овие инстанци без достигнување на временскиот лимит. Timeout записите кај backtracking се цензурирани на 10 секунди, па не треба да се толкуваат како вистинско време потребно за завршување.

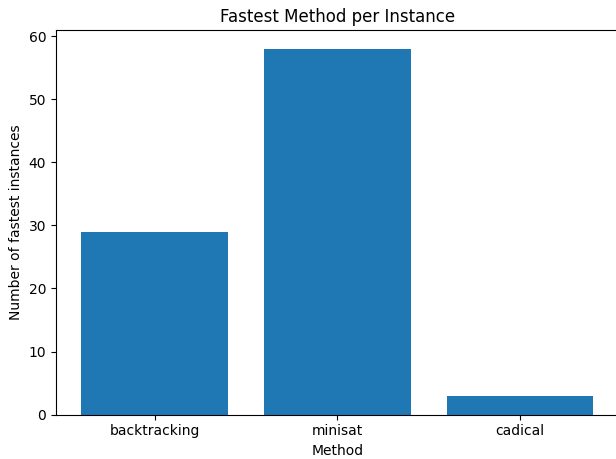
6.4 Најбрз метод по инстанца

За секоја од 90-те инстанци е определен методот со најмало вкупно време на извршување. Резултатите се:

- Backtracking е најбрз кај 29 инстанци;
- MiniSat е најбрз кај 58 инстанци;

- CaDiCaL е најбрз кај 3 инстанци.

Распределбата е прикажана на Слика 3.



Слика 3: Број на инстанци во кои секој метод има најмало вкупно време.

MiniSat има најголем број победи во конкретната експериментална поставеност. Ова не претставува општ заклучок дека MiniSat е подобар од CaDiCaL за произволни SAT проблеми. Споредбата важи за ова енкодирање, овие големини на граф, избраните параметри и конкретната софтверска и хардверска околина.

6.5 Големината на SAT енкодирањето

Со зголемување на бројот на темиња растат и бројот на SAT променливи и бројот на CNF клаузули. Бројот на променливи го вклучува и sequential-counter помошниот простор, а бројот на клаузули ги вклучува и edge ограничувањата и cardinality енкодирањето.

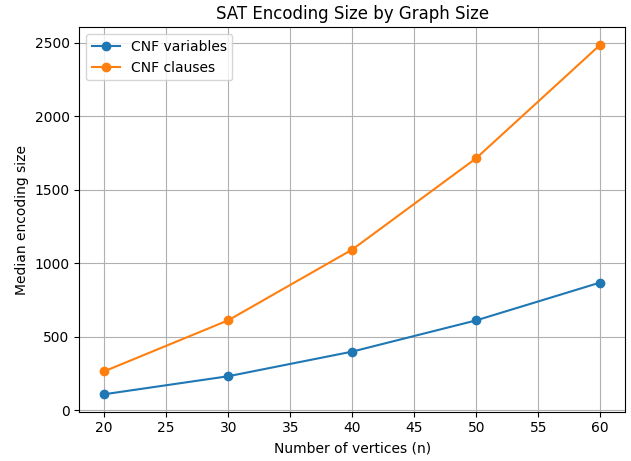
Табела 3: Медијална големина на SAT енкодирањето

n	CNF променливи	CNF клаузули
20	110	266
30	232.5	613
40	400	1094
50	612.5	1715.5
60	870	2487

Бројот на помошни променливи зависи од n и k , додека бројот на клаузули дополнително зависи и од конкретно генерираните ребра. Поради тоа во табелата се прикажани медијални вредности за сите тестирани инстанци со ист број темиња.

Растот на големината на SAT формулата е прикажан на Слика 4.

Иако големината на енкодирањето расте со n , SAT решавачите остануваат ефикасни за разгледуваните инстанци. Самата големина на CNF, сепак, не е доволна за предвидување на solver времето: тежината зависи и од структурата на формулата и од тоа колку брзо solver-от може да изведе пропагација и да најде модел или доказ за незадоволност.



Слика 4: Раст на бројот на CNF променливи и клаузули со бројот на темиња.

6.6 Главни согледувања

Од финалните резултати може да се издвојат следните согледувања:

- кај најмалите инстанци backtracking може да биде побрз поради малиот overhead на директното пребарување;
- со раст на n SAT пристапот покажува постабилно скалирање во рамките на тестираниот опсег;
- ниската вредност $p = 0.2$ создава особено тешки инстанци за конкретната backtracking имплементација, додека кај $p = 0.8$ директниот пристап повторно станува конкурентен;
- сите 10 timeout случаи на backtracking се концентрирани кај $p = 0.2$ и $n \in \{50, 60\}$;
- MiniSat има најмало вкупно време кај 58 од 90 инстанци, но резултатот е специфичен за оваа експериментална поставеност;
- големината на CNF расте со n , но во тестираниот опсег тоа не го спречува SAT пристапот да остане практично ефикасен.

Оттука, експериментите не поддржуваат тврдење дека еден метод е универзално најдобар. Тие покажуваат дека релативната предност зависи од големината и структурата на конкретната инстанца, како и од трошокот за енкодирање и од однесувањето на избраниот SAT решавач.

7 Заклучок

Во овој проект е направена експериментална споредба меѓу директен backtracking алгоритам и SAT-базиран пристап за decision верзијата на Independent Set. Кај SAT пристапот истата CNF формула, составена од ограничувања за независност и sequential-counter енкодирање на условот $|S| \geq k$, се решава со MiniSat и CaDiCaL.

Финалните 90 инстанци покажуваат јасен crossover во однесувањето на методите. За $n = 20$ backtracking има најниско медијално време, додека веќе од $n = 30$ MiniSat станува благо побрз во агрегираниот резултат. Кај $n = 60$ медијалното време на backtracking е приближно 9.25 пати поголемо од медијалното вкупно време на MiniSat.

Параметарот p има уште поизразено влијание врз директното пребарување. Кај $p = 0.2$ backtracking има медијално време од околу 1.385 секунди, наспроти 0.01345 секунди за MiniSat, а сите 10 backtracking timeout случаи се појавуваат кај $p = 0.2$ и $n \in \{50, 60\}$. Кај $p = 0.8$, пак, backtracking е благо побрз од MiniSat во агрегираниот медијален резултат. Ова покажува дека структурата на графот може суштински да ја промени практичната тежина на истата NP-комплетна decision задача.

MiniSat има најмало вкупно време кај 58 од 90 инстанци, Backtracking кај 29, а CaDiCaL кај 3. Овие броеви не треба да се генерализираат како универзален ранг на алгоритмите: тие се резултат на конкретното Independent Set енкодирање, sequential-counter cardinality ограничувањето, избраните параметри и мерната околина.

Анализата на CNF големината потврдува дека бројот на променливи и клаузули расте со n , но специјализираните SAT решавачи и понатаму ефикасно ги решаваат формулите во разгледуваниот опсег. Практичната предност на SAT пристапот затоа не произлегува од помал простор на можни решенија, туку од тоа што современите SAT решавачи користат високо оптимизирани механизми за пропација, пребарување и учење врз структурата на CNF формулата.

Главниот заклучок е дека редукцијата во SAT претставува практично силна алтернатива на директното комбинаторно пребарување, особено за поголемите и потешките инстанци во овој експеримент. Истовремено, резултатите ја нагласуваат потребата алгоритмите да се споредуваат по семејства на инстанци, а не само преку една глобална просечна вредност.

Литература

- [1] R. M. Karp, *Reducibility Among Combinatorial Problems*, in *Complexity of Computer Computations*, pp. 85–103, 1972.
- [2] N. Eén and N. Sörensson, *An Extensible SAT-solver*, Theory and Applications of Satisfiability Testing (SAT 2003), Lecture Notes in Computer Science, vol. 2919, pp. 502–518, 2003.
- [3] A. Biere, *CaDiCaL at the SAT Race 2019*, Proceedings of SAT Race 2019: Solver and Benchmark Descriptions, vol. B-2019-1, pp. 14–15, 2019.
- [4] A. Ignatiev, A. Morgado and J. Marques-Silva, *PySAT: A Python Toolkit for Prototyping with SAT Oracles*, Theory and Applications of Satisfiability Testing (SAT 2018), Lecture Notes in Computer Science, vol. 10929, pp. 428–437, 2018.
- [5] PySAT Documentation, *Cardinality Encodings* (*pysat.card*), documentation for the CardEnc class and AtLeastK encoding.
- [6] C. Sinz, *Towards an Optimal CNF Encoding of Boolean Cardinality Constraints*, Principles and Practice of Constraint Programming (CP 2005), pp. 827–831, 2005.